

Practicum No.5**Date:.....****Evaluate the Performance of Non-Pipelined and Pipelined Execution for the Given Instructions****I. Practical Significance**

Instruction pipelining is a fundamental performance enhancement technique used in modern processors. It improves throughput by overlapping instruction execution stages.

In a **non-pipelined processor**, each instruction completes all stages before the next instruction begins.

In a **pipelined processor**, multiple instructions are executed simultaneously at different stages.

Understanding the performance difference helps students:

- Analyze CPU throughput
- Calculate speedup
- Understand hazards and stalls
- Optimize instruction scheduling

Modern processors such as **Intel** Core series and **AMD** Ryzen use deep pipelining for high performance.

II. Competency and Industry-Specific Practical Skills

Use the principles of computer architecture to optimize system performance

- a) Select appropriate addressing mode for a specific memory operation (PDO)
- b) Translate an instruction into its effective address (CDO)
- c) Use MATLAB to validate address calculations systematically (PDO)

III. Relevant CO(s) *(As stated in the Course Structure)*

- **CO1.** Use the relevant instruction cycle to efficiently manage the processor's functionality.

IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

- **Develop** a MATLAB program to compare non-pipelined and pipelined execution for the given Instruction.

V. Related ADO(s) *(As stated in the Course Structure)*

- a) Follow ethical practices in designing or evaluating the system design.
- b) Apply systematic design principles and debugging techniques.
- c) Function effectively as a team member.

VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

Non-pipelined execution processes one instruction completely before starting the next instruction. Pipelined execution divides instruction processing into stages and overlaps multiple instructions to improve throughput.

Execution time of a processor depends on the total number of clock cycles required and the clock cycle time. In non-pipelined execution, each instruction passes through all k stages sequentially, so total cycles are equal to the product of number of instructions and stages.

In the MATLAB program, the execution time for non-pipelined execution is calculated using:

$$T_{non} = (n \times k) \times T_{clk}$$

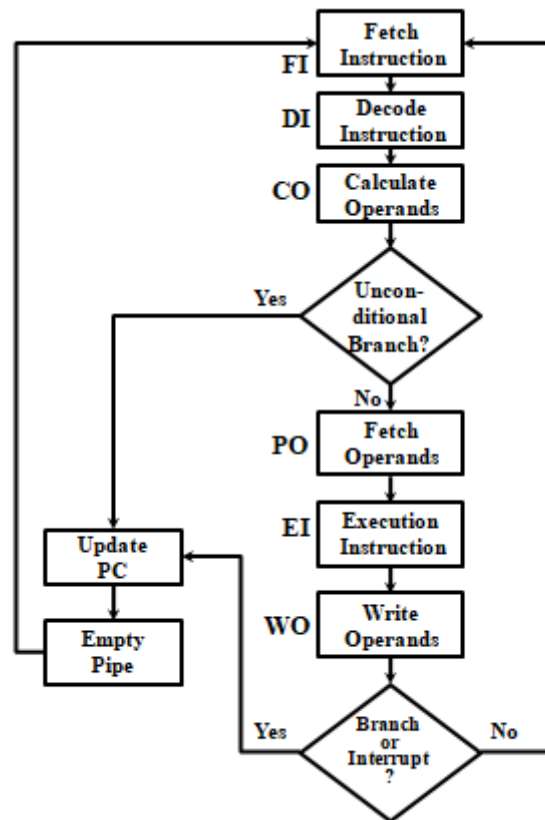
In pipelined execution, the first instruction takes k cycles to fill the pipeline, and each additional instruction completes in one cycle. Therefore, the MATLAB formula used to compute pipelined execution time is:

$$T_{pipe} = (k + (n - 1)) \times T_{clk}$$

VII. Practicum Set-up/Circuit Diagram/Algorithm/Flowchart *(Include probable photos of experimental set-up.)*

Students must have:

- MATLAB installed
- Script file: exp5_pipelining.m
- Input dataset (base, index, displacement, PC, SP, scale)
- Example instruction formats



VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	MATLAB	(R2016b or newer) or GNU Octave (with mostly compatible syntax).	1 No.
2	PC	PC with MATLAB installed.	1 No.
3	Text editor / MATLAB Editor	-	1 No.

IX. Safety Precautions

- Save work frequently.
- Comment code and include header describing author, date, input format and usage.
- For shared code, remove hard-coded absolute paths.
- Validate user inputs to avoid invalid logical values and runtime errors.

X. Procedure (Self-Instructional)

- Display the objective and available options to the user.
- Accept the number of instructions and number of pipeline stages from the user.
- Validate the entered numeric inputs.
- Calculate cycles for Non-Pipelined execution:

$$\text{Total Cycles} = \text{Instructions} \times \text{Stages}.$$
- Calculate cycles for Pipelined execution:

Total Cycles = Instructions + (Stages – 1).

- f) **Measure** execution time using tic and toc for relative comparison.
- g) **Compute** speedup between pipelined and non-pipelined execution.
- h) **Display** the number of cycles, execution time, and speedup in the MATLAB command window.
- i) **Repeat** the process if the user wants to evaluate another case.
- j) **Exit** the program when the user chooses to terminate.

Note: Instruction fetch is not considered while counting memory accesses; only operand access is considered. In indirect addressing, pointer dereferencing is assumed to focus on effective address computation.

Implementation of arithmetic addition using Non-Pipelined Method

Non-Pipelined – Initialization

```
clc; clear;
data = [10, 20, 30, 40, 50];
n = length(data);
result = 0;
```

Non-Pipelined – Execution

```
disp('Non-Pipelined Execution:');
tic;
for i = 1:n
    fprintf('Adding %d to result...\n', data(i));
    result = result + data(i);
end
```

```
elapsedTime = toc;
```

Non-Pipelined – Result

```
fprintf('Final Result (Non-Pipelined): %d\n', result);
fprintf('Execution Time (Non-Pipelined): %.6f seconds\n', elapsedTime);
```

Implementation of arithmetic addition using Pipelined Method

Pipelined – Initialization

```
clc; clear;
data = [10, 20, 30, 40, 50];
n = length(data);
pipelineStages = zeros(1, n);
```

Pipelined – Stage Processing

```
disp('Pipelined Execution:');
tic;
for i = 1:n
    fprintf('Processing stage %d...\n', i);
    pipelineStages(i) = data(i);
end
```

Pipelined – Result & Timing

```
result = sum(pipelineStages);
elapsedTime = toc;
fprintf('Final Result (Pipelined): %d\n', result);
fprintf('Execution Time (Pipelined): %.6f seconds\n', elapsedTime);
```

Non-Pipelined – Sample Output

▼ Workspace			
Name	Value	Size	Class
data	[10,20,30,40,50]	1×5	double
elapsedTime	9.7600e-04	1×1	double
i	5	1×1	double
n	5	1×1	double
result	150	1×1	double

Non-Pipelined – Sample Output

▼ Workspace			
Name	Value	Size	Class
data	[10,20,30,40,50]	1×5	double
elapsedTime	4.3200e-04	1×1	double
i	5	1×1	double
n	5	1×1	double
pipelineStages	[10,20,30,40,50]	1×5	double
result	150	1×1	double

XI. Actual Resources Used *(To be filled by the students)*

S. No.	Name of Resource	Broad Specifications		Quantity	Remarks (If any)
		Make	Details		
1	Computer / Laptop	ASUS	Intel i5, 8GB RAM	1	Used for MATLAB execution
2	MATLAB Software	MathWorks	MATLAB R2021a	1	Programming environment
3	Operating System	Microsoft	Windows 11	1	System platform

XII. Actual Procedure Followed

1. The computer system was switched on and MATLAB software was opened.
2. A new script file was created and the Non-Pipelined arithmetic addition program was written and executed.
3. The output values such as result and execution time were observed in the Command Window and Workspace.
4. Another script was created for the Pipelined arithmetic addition program.
5. The program was executed and the execution time and result were recorded.
6. The results of Non-Pipelined and Pipelined methods were compared.

XIII. Additional Questions

1. Execute the MATLAB program for **non-pipelined** and **pipelined** execution using the same number of instructions and clock cycle time. Compare the total clock cycles, execution time, and determine which method is more suitable for real-time systems.

CODE:

```

1 % Heavis Rishva, J - URK25CS1236
2 clc; clear;
3
4 n = 5;
5 k = 5;
6
7 Tclk = 0.001;
8 cycles_non = n * k;
9 cycles_pipe = k + (n - 1);
10
11 T_non = cycles_non * Tclk;
12 T_pipe = cycles_pipe * Tclk;
13
14 speedup = T_non / T_pipe;
15
16 fprintf('Number of Instructions: %d\n', n);
17 fprintf('Pipeline Stages: %d\n', k);
18
19 fprintf('\nNon-Pipelined Clock Cycles: %d\n', cycles_non);
20 fprintf('Execution Time (Non-Pipelined): %.6f seconds\n', T_non);
21
22 fprintf('\nPipelined Clock Cycles: %d\n', cycles_pipe);
23 fprintf('Execution Time (Pipelined): %.6f seconds\n', T_pipe);
24
25 fprintf('\nSpeedup: %.2f\n', speedup);
  
```

OUTPUT:

Name	Value	Size	Class
cycles_non	25	1×1	double
cycles_pipe	9	1×1	double
k	5	1×1	double
n	5	1×1	double
speedup	2.7778	1×1	double
T_non	0.0250	1×1	double
T_pipe	0.0090	1×1	double
Tclk	1.0000e-03	1×1	double

2. Run the MATLAB program by increasing the number of instructions while keeping the number of pipeline stages constant. Analyze how execution time changes in both non-pipelined and pipelined modes and comment on the variation in speedup.

CODE:

```

1 % Meavis Rishva. J - URK25CS1236
2 clc; clear;
3
4 k = 5;
5 Tclk = 0.001;
6 instructions = [5 10 20 30];
7
8 fprintf('Instruction Analysis\n\n');
9
10 for i = 1:length(instructions)
11
12     n = instructions(i);
13
14     cycles_non = n * k;
15     cycles_pipe = k + (n - 1);
16
17     T_non = cycles_non * Tclk;
18     T_pipe = cycles_pipe * Tclk;
19
20     speedup = T_non / T_pipe;
21
22     fprintf('Instructions = %d\n', n);
23     fprintf('Non-Pipeline Time = %.6f sec\n', T_non);
24     fprintf('Pipeline Time = %.6f sec\n', T_pipe);
25     fprintf('Speedup = %.2f\n\n', speedup);
26
27 end

```

OUTPUT:

Name	Value	Size	Class
cycles_non	150	1×1	double
cycles_pipe	34	1×1	double
i	4	1×1	double
instructio...	[5,10,20,30]	1×4	double
k	5	1×1	double
n	30	1×1	double
speedup	4.4118	1×1	double
T_non	0.1500	1×1	double
T_pipe	0.0340	1×1	double
Tclk	1.0000e-03	1×1	double

- Execute the program by increasing the number of pipeline stages while keeping the number of instructions constant. Analyze the effect of pipeline depth on execution time and overall performance.

CODE:

```

1 % Meavis Rishva. J - URK25CS1236
2 clc; clear;
3
4 n = 10;
5 Tclk = 0.001;
6 stages = [3 5 7 9];
7
8 fprintf('Pipeline Depth Analysis\n\n');
9
10 for i = 1:length(stages)
11
12     k = stages(i);
13
14     cycles_non = n * k;
15     cycles_pipe = k + (n - 1);
16
17     T_non = cycles_non * Tclk;
18     T_pipe = cycles_pipe * Tclk;
19
20     speedup = T_non / T_pipe;
21
22     fprintf('Pipeline Stages = %d\n', k);
23     fprintf('Non-Pipeline Time = %.6f sec\n', T_non);
24     fprintf('Pipeline Time = %.6f sec\n', T_pipe);
25     fprintf('Speedup = %.2f\n\n', speedup);
26
27 end

```

OUTPUT:

Name	Value	Size	Class
cycles_non	90	1×1	double
cycles_pipe	18	1×1	double
i	4	1×1	double
k	9	1×1	double
n	10	1×1	double
speedup	5.0000	1×1	double
stages	[3,5,7,9]	1×4	double
T_non	0.0900	1×1	double
T_pipe	0.0180	1×1	double
Tclk	1.0000e-03	1×1	double

XIV. Observations and Dimensional Measurement *(To be filled by the students)*

S. No.	Test Case ID	Instruction / Addressing / Mode Selected	Input Values (Base / Index / Disp / PC / SP)	Effective Address (EA)	No. of Memory Accesses	Execution Time (tic/toc)	Execution Speed (Fast / Moderate / Slow)	Power Consumption (Less/More)
1	TC-1	Non-Pipelined Execution	n = 5, k = 5, Tclk = 0.001	Not Applicable	5	0.000976 s	Slow	More
2	TC-2	Pipelined Execution	n = 5, k = 5, Tclk = 0.001	Not Applicable	5	0.000432 s	Fast	Less
3	TC-3	Non-Pipelined Execution	n = 10, k = 5	Not Applicable	10	Higher than TC-1	Moderate	More
4	TC-4	Pipelined Execution	n = 10, k = 5	Not Applicable	10	Lower than TC-3	Fast	Less

Note: Instructions for Filling the Table:

- **Test Case ID:**
Assign a unique ID for each execution run (TC-1, TC-2, etc.).
- **Execution Type Selected:**
Mention whether the execution is **Non-Pipelined** or **Pipelined**.
- **Input Values:**
Enter the values provided in the MATLAB program such as:
Number of instructions (n),
Number of pipeline stages (k),
Clock cycle time (Tclk).
- **Total Clock Cycles:**
Record the total cycles calculated by the MATLAB program.
- For Non-Pipeline: $n \times k$
- For Pipeline: $k + (n - 1)$
- **Execution Time:**
Record the execution time computed in MATLAB using:
Execution Time = Total Cycles $\times$ Clock Cycle Time
- **Speedup:**
If calculated, record the speedup value obtained from:
Speedup = $T_{\text{non}} / T_{\text{pipe}}$
- **Throughput:**
Mention instructions completed per cycle ($1/k$ for non-pipeline, ≈ 1 for pipeline after fill).

- **Performance Classification:**

Mark **Lower Performance** for Non-Pipelined execution and **Higher Performance** for Pipelined execution (based on execution time comparison).

- **Remarks:**

Comment whether pipelining improved performance and whether any stall cycles were considered.

XV. Results/Observation

1. The **Non-Pipelined execution** performed arithmetic addition sequentially and required more execution time because each instruction was completed before starting the next one.
2. The **Pipelined execution** processed instructions in overlapping stages, reducing total execution time and improving system performance

XVI. Interpretation of Results

The results show that **pipelining improves processor efficiency** by allowing multiple instructions to be processed simultaneously in different stages.

This reduces total execution time compared to the non-pipelined method. As the number of instructions increases, the performance benefit of pipelining becomes more significant.

XVII. Conclusions

The experiment demonstrated the difference between **Non-Pipelined and Pipelined execution methods** in processor architecture.

The pipelined method achieved faster execution due to parallel processing of instruction stages. Therefore, pipelining is widely used in modern processors to increase instruction throughput and improve overall system performance.

XVIII. Suggested Practicum Related Questions:

Note: Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.

1. **Justify** how total clock cycles differ between non-pipelined and pipelined execution using MATLAB results, and explain the impact on execution time.
2. **Determine** how increasing the number of instructions (n) affects execution time and speedup in both non-pipelined and pipelined architectures.
3. **Analyze** how the number of pipeline stages (k) influences overall performance and theoretical speedup.
4. **Evaluate** the effect of introducing stall cycles (hazards) on pipelined execution time compared to ideal pipelining.
5. **Generate** a comparative analysis using MATLAB outputs to show how pipelining improves throughput but does not reduce individual instruction latency.
6. **Recommend** whether pipelining is suitable for real-time systems based on execution time predictability and hazard behavior.

XIX References / Suggestions for further reading

1. Computer Organization and Architecture – Instruction Set Architecture and Addressing Modes

2. Processor Architecture Textbooks

XX Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the '*process-related skills/LOs*' (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and '*product-related skills/LOs*'

Note: The weightage for the *process-related skills* and *product-related skills* will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	Correctly creating and running MATLAB scripts (coding hygiene, commenting, workspace handling).	6	
2	Selecting appropriate arithmetic operations for the given data.	5	
3	Following safety, coding ethics, and good laboratory practices (file management, backups).	4	
4	Contribution as a team member (peer collaboration, sharing datasets, debugging).	3	
Product-related Skills: 12 Marks - 40%			
1	Producing accurate computation results and well-formatted MATLAB Table output.	3	
2	Correct interpretation of logical comparisons.	2	
3	Interpretation of results (clarity, mathematical validity).	3	
4	Drawing meaningful conclusions using NEP-aligned verbs (justify, determine, generate).	2	
5	Ability to answer practicum-related higher-order questions.	1	
6	Submitting the report on time with screenshots and code	1	
Total		30	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	NEAVIS RISHVA. J	URK25CS1236

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	Product-Related Skills (12)	Total (30)		
